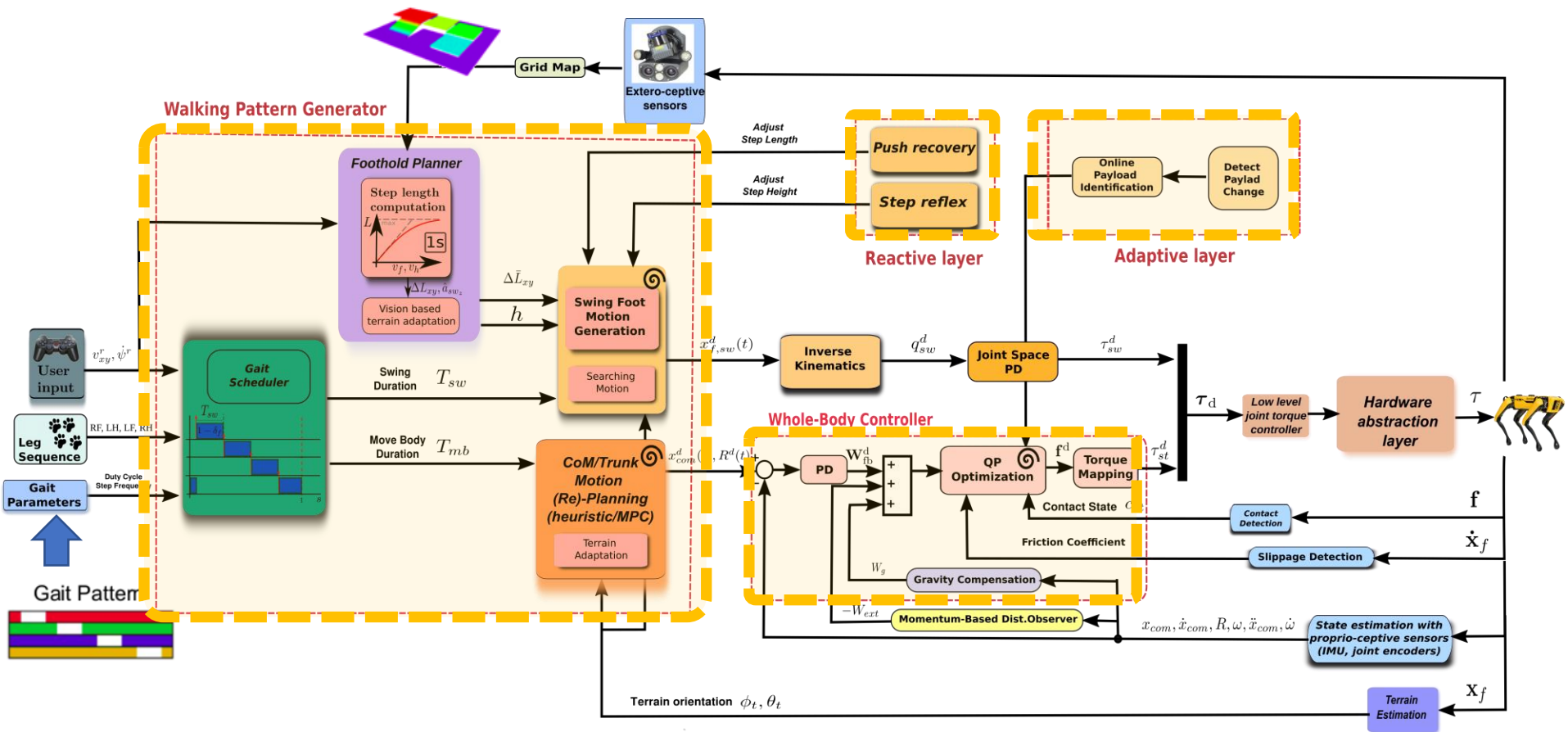


# Legged Locomotion

**Prof. Michele Focchi, Associate Professor**  
University of Trento

# Heuristic Locomotion Framework



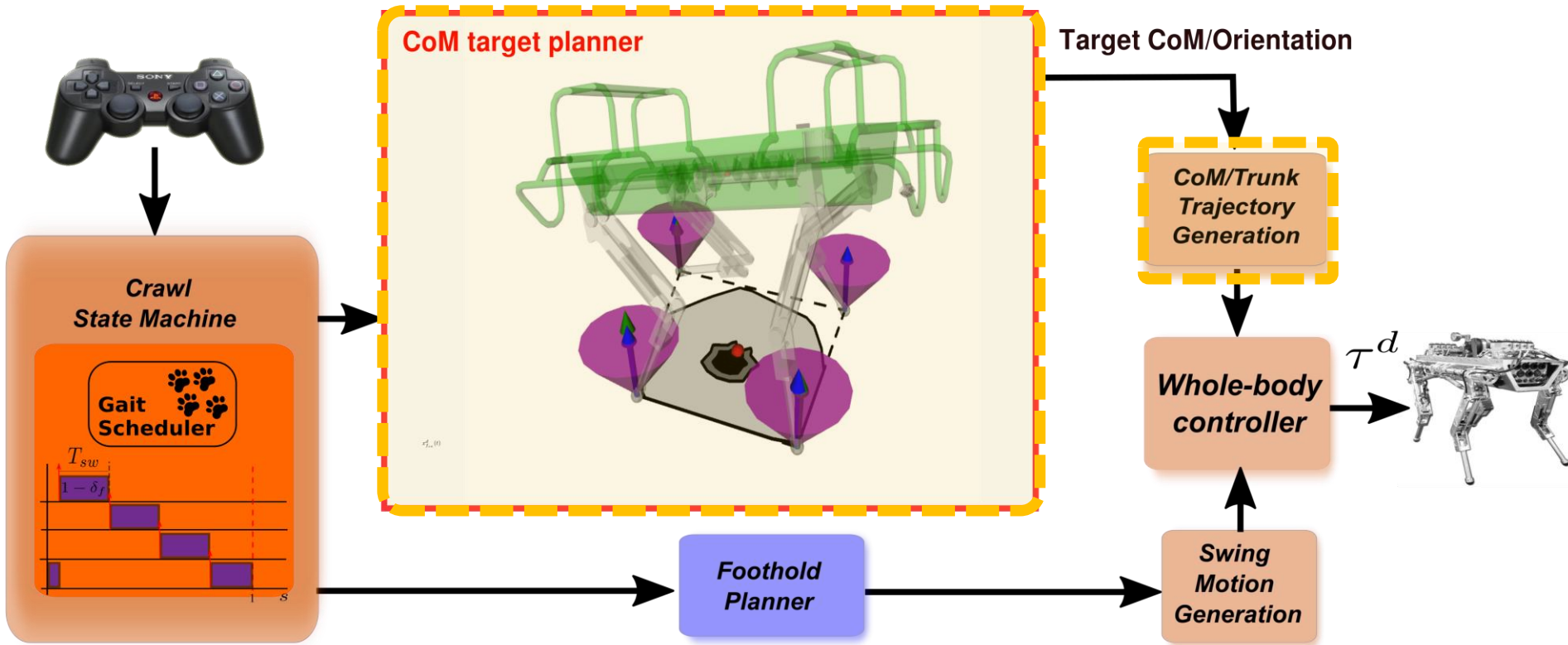
① Motion Planner

② Whole-body controller

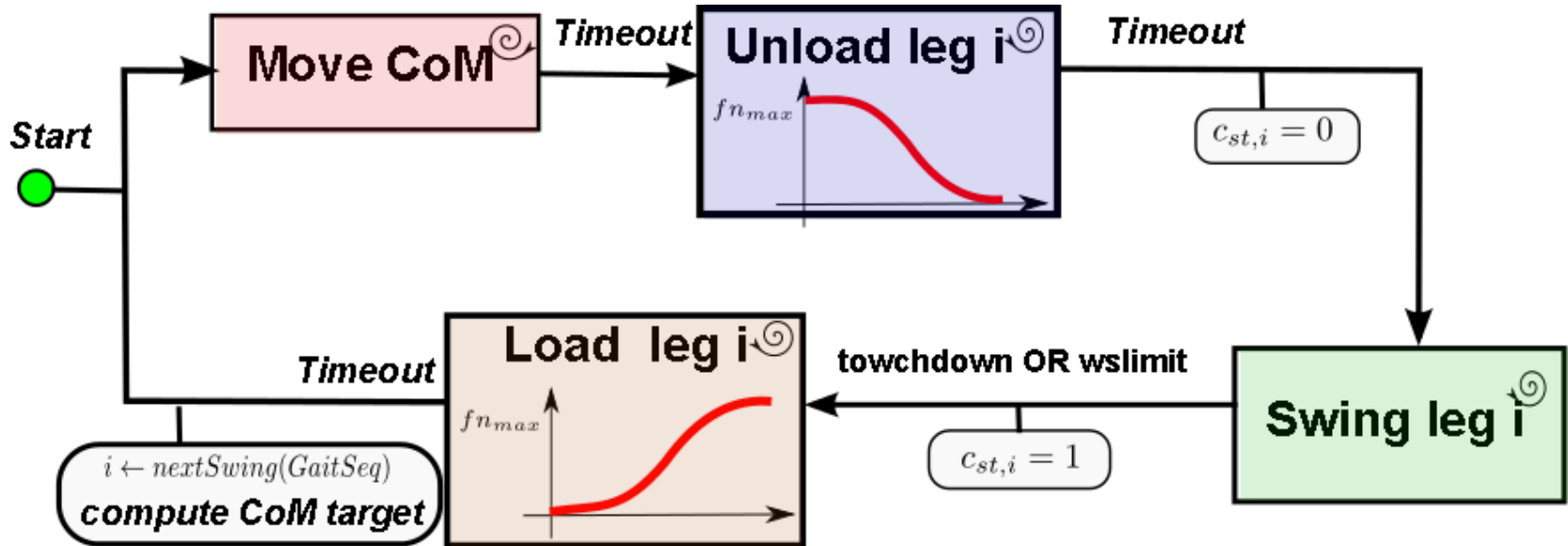
③ Reactive Layer

④ Adaptive Layer

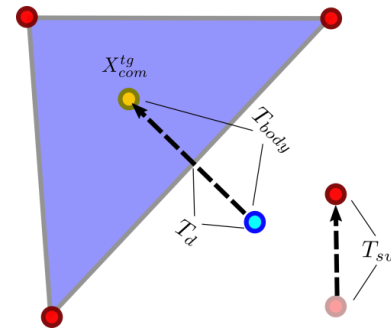
# Heuristic Locomotion Framework Simplified



# Heuristic Static Crawl : State machine



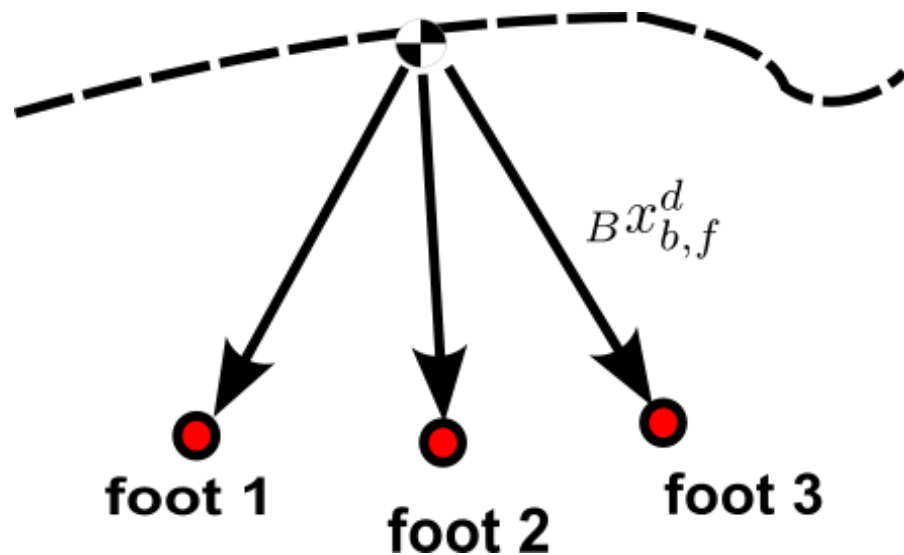
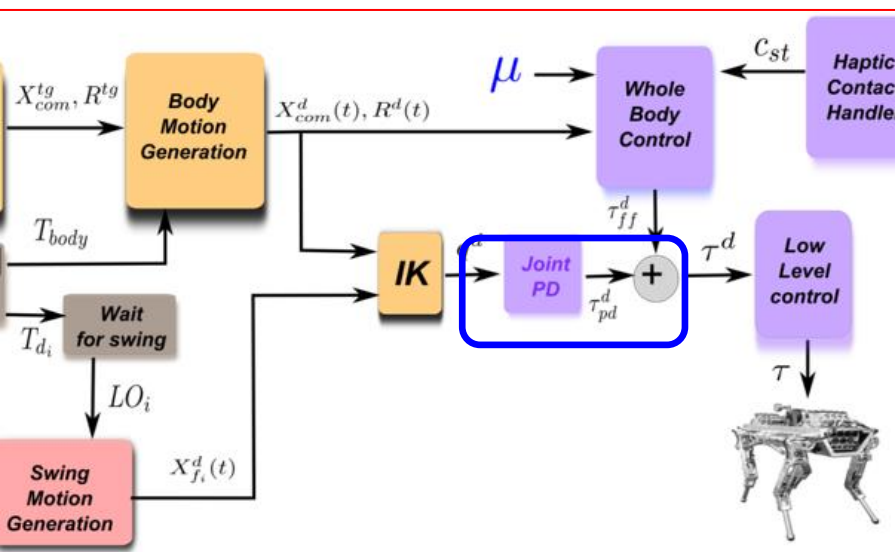
- ① Move CoM inside **support triangle** formed by the 3 legs opposite to the swing one (static stability) (move CoM phase)
- ② Spline upper bound on normal force down to zero (unload phase)
- ③ Swing motion + contact searching motion (swing phase, reactive)



- ④ Gradually redistribute weight on 4 legs (load phase)

# Foot trajectory - Stance

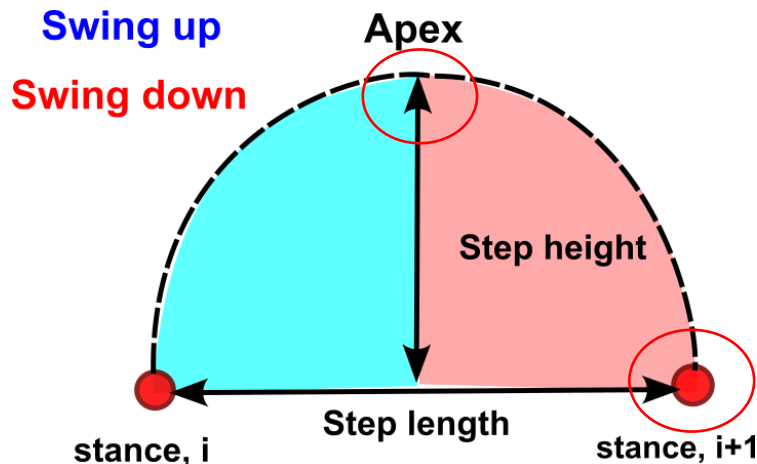
- What about the foot trajectory? In stance is **constant**
- In stance **ideally** we don't need joint PD control (we have trunk controller)
- in **practice**, for robustness in case of foot slippage, is good to have a reference of a PD control running in parallel
- Therefore we compute the foot position relative to CoM and perform IK



$$Bx_{b,f}^d = {}_B R_W ({}_W x_{f,st} - {}_W x_{com}^d) \rightarrow \mathbf{IK} \rightarrow q^{des}$$

# Foot trajectory - swing

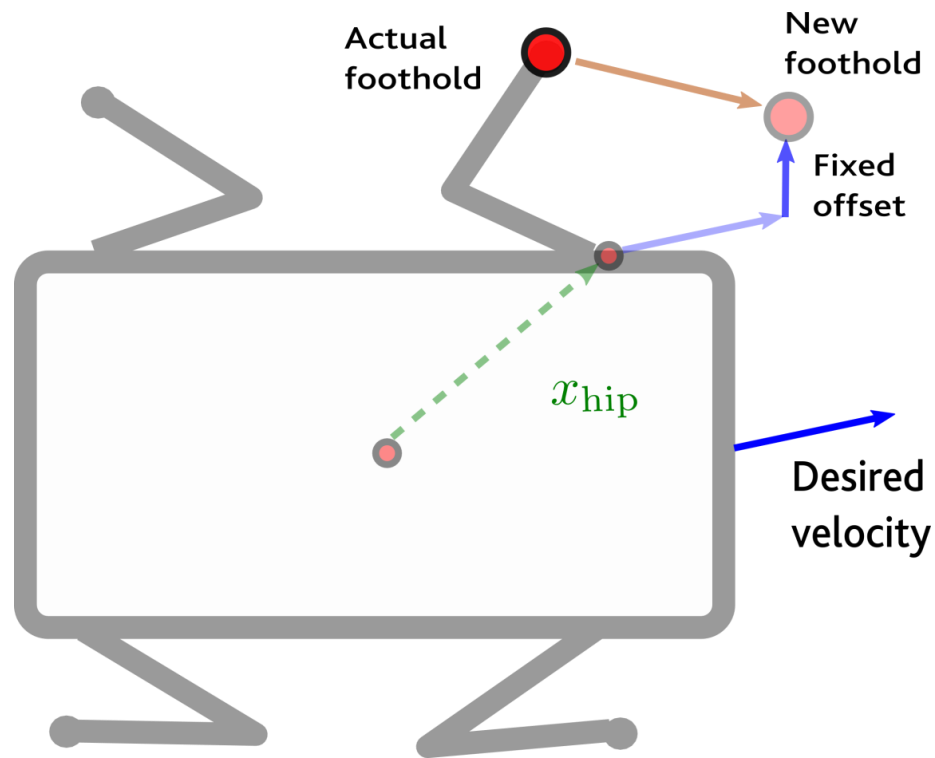
- If swing trajectory is not optimized typically polynomials are used
  - 3rd order → enforce position/velocity initial and final conditions
  - 5th order → enforce position/velocity/acceleration initial and final conditions
- Typically: initial/final velocity and acceleration are zero.



- **horizontal coordinates (X-Y):** 1 polynomial
$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3.$$
- **vertical coordinate Z:** split into 2 polynomials with a midpoint. The midpoint is used to impose the desired **step height**.
- How to plan the foothold?

## “Robo-centric” foothold computation

- compute the first foothold about the position of the hip rather than with respect to the **previous** foot position
- Crucial to avoid progressive degeneration of the support polygon...



## “ Raibert-Style “ foothold planning

[Raibert, 1986]

- Heuristic approaches are not based on a mathematical formulation with formal proofs but rather on experience, **intuition** and practice.
- **User inputs:** 2D linear velocities (x,y), angular rate, and gait pattern
- Raibert defined the **neutral point**, i.e., the point about which the hopper has a symmetric motion of the body.

$$\mathbf{p}_{i,\text{nom}} = \mathbf{p}_{i,\text{hip}} + \underbrace{\mathbf{v}_{\text{com, meas}} \frac{T_{\text{st}}}{2}}_{\text{Compensation due to average velocity}} + k \underbrace{(\mathbf{v}_{\text{com, meas}} - \mathbf{v}_{\text{com}}^d)}_{\text{Error correction}}$$

Nominal Foothold locations

Compensation due to average velocity

Error correction

- $\mathbf{p}_{\text{nom}} \in \mathbb{R}^2$  is the position of the neutral point on an horizontal plane
- $\mathbf{p}_{\text{hip}} \in \mathbb{R}^2$  is the projection of the hip on the ground
- $T_{\text{st}} \in \mathbb{R}$  is the duration of the stance phase
- $\mathbf{v}^d \in \mathbb{R}^2$  is the X-Y components of desired CoM velocity

## Example Raibert-style foothold planning

[M.Raibert, Legged Robots That Balance, 2000]



# Rough terrain locomotion – terrain adaptation

[Focchi et al., STAR 2020]

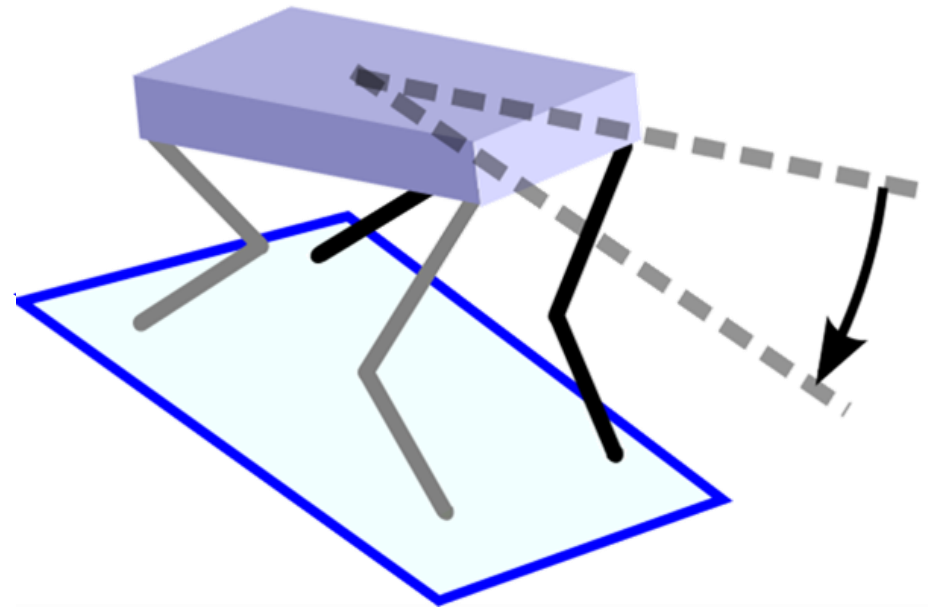
- Differently from flat terrain a rough terrain requires an **adaptation strategy**:
  - ① Adapt trunk orientation to terrain inclination (reduces risk to reach kinematic limits)

- ② Haptic touch-down

- ③ Online re-planning

- Reference for trunk orientation?

- Estimate terrain plane inclination
- Align the trunk with that plane, (consider some temporal filtering)
- Or...optimize also the trunk orientation (SRBD/Centroidal dynamics)



# The horizontal frame

- To estimate terrain plane  $\Pi$  we need to define feet positions in the **horizontal frame**

- horizontal frame** ( $H$ ) is like base frame but with the  $Z$  axis aligned with gravity

- The rotation matrix is mapping from  $H$  to  $W$  is

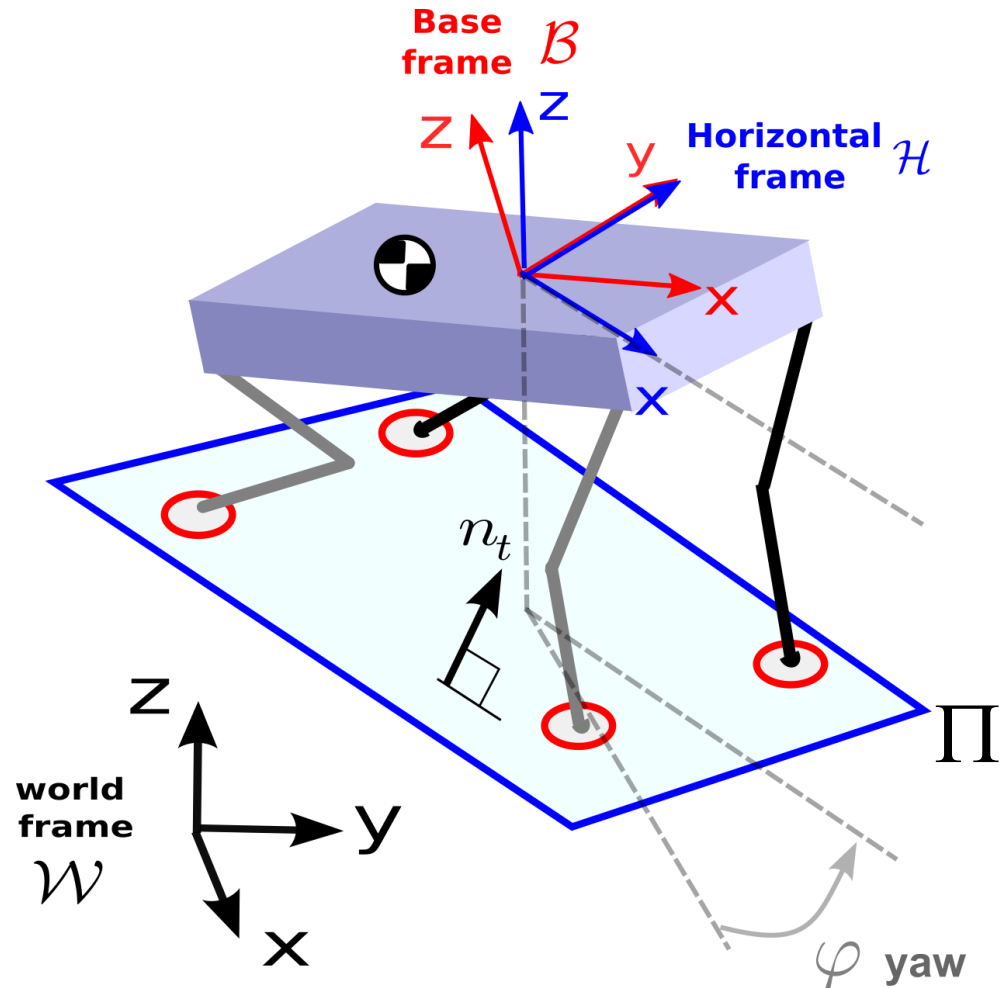
$${}_WR_H = R_z(\varphi),$$

- The rotation matrix is mapping from  $B$  to  $H$  is

$${}_HR_B = R_y(\theta)R_x(\psi),$$

pitch

roll



## Terrain estimator for trunk orientation adaptation

- Terrain plane inclination is typically estimated by **fitting** a plane through the stance feet
- The fitting plane can be found minimizing the “distance” along the Z component between the fitting plane ( $\Pi : \mathbf{a}x + \mathbf{b}y + \mathbf{c}z + \mathbf{d} = 0$ ) and the set of points represented by the feet positions (in HF)

$$A = \begin{bmatrix} H\mathcal{X}f_{1x} & H\mathcal{X}f_{1y} & 1 \\ H\mathcal{X}f_{2x} & H\mathcal{X}f_{2y} & 1 \\ H\mathcal{X}f_{3x} & H\mathcal{X}f_{3y} & 1 \\ H\mathcal{X}f_{4x} & H\mathcal{X}f_{4y} & 1 \end{bmatrix}, \quad b = \begin{bmatrix} -H\mathcal{X}f_{1z} \\ -H\mathcal{X}f_{2z} \\ -H\mathcal{X}f_{3z} \\ -H\mathcal{X}f_{4z} \end{bmatrix} \quad \Rightarrow \quad x = \begin{bmatrix} a \\ b \\ d \end{bmatrix} = A^\# b,$$

- Then, the normal to the terrain plane  $n_t$  and the corresponding pitch/roll angles  $\theta_t / \phi_t$  (Hp ZYX convention), can be obtained as

$$n_t = \begin{bmatrix} a & b & 1 \end{bmatrix}^T / \left\| \begin{bmatrix} a & b & 1 \end{bmatrix} \right\|,$$

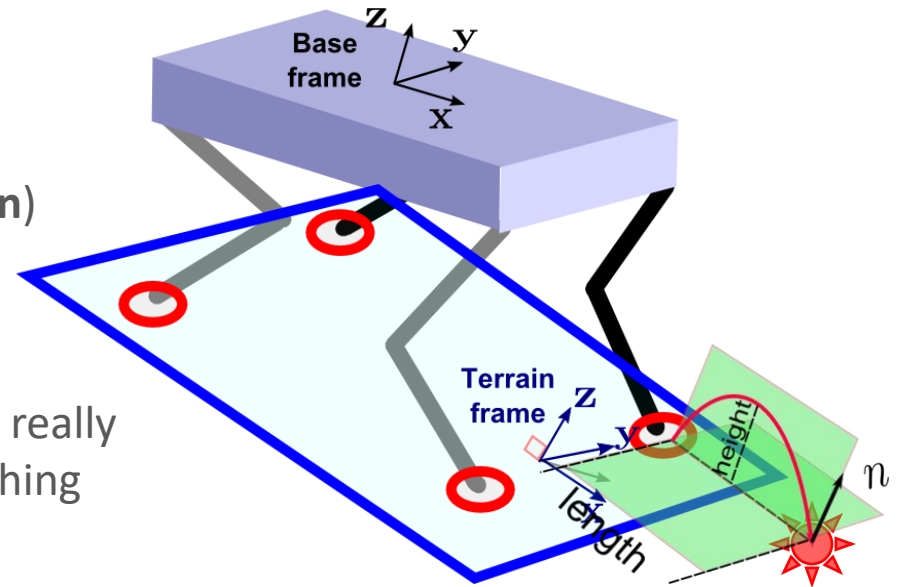
$$\theta_t = \text{atan} (n_{tx} / n_{tz}),$$

$$\phi_t = \text{atan} (-n_{ty} \sin(\theta_t) / n_{tx}).$$

- The terrain estimate should be updated at each change of contact state
- Works with (at least) 3 non-collinear points, with trot update only at 4 stance (otherwise A rank def.)
- Base pose reference is adapted to local terrain estimation (pitch/roll) + nominal height

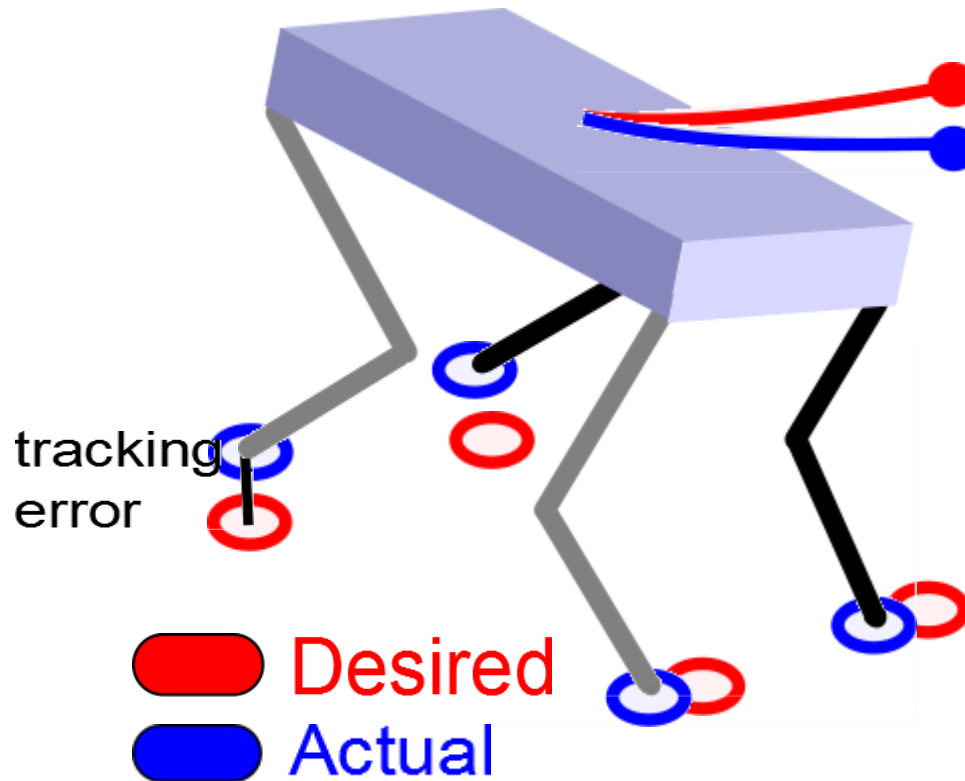
# Haptic touch-down

- Avoids creating destabilizing forces due to discrepancies between the plan and the real environment
- Swing plane is aligned with the terrain plane
- Check ground reaction forces only during the swing-down phase (**haptic touchdown**)
- Stop swing trajectory when the contact is really achieved (premature touch-down or reaching motion)
- Lift-off instant is given by the planner to keep synchronism



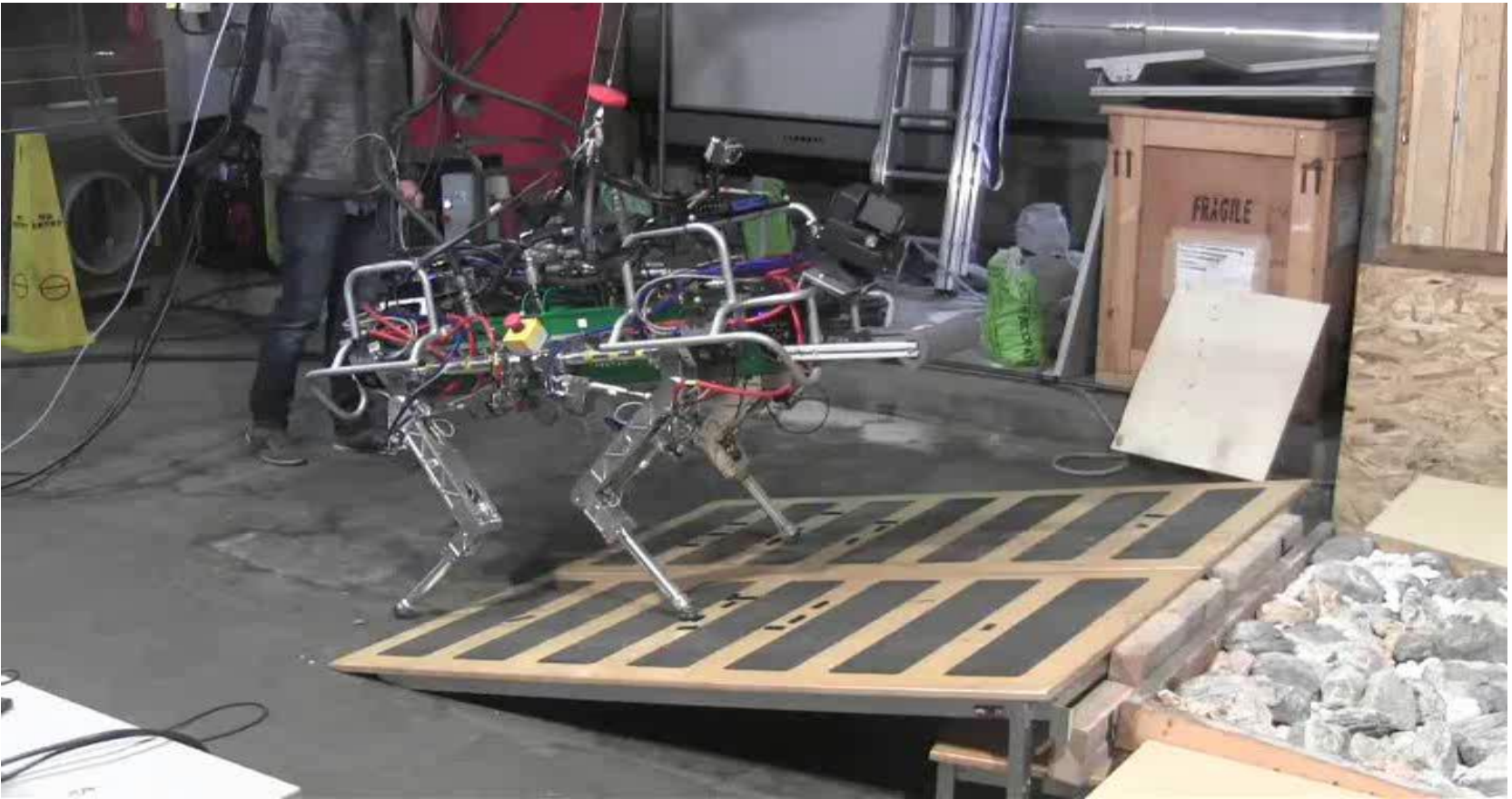
# Online CoM/trunk trajectory re-planning

- recover tracking errors due to: disturbances, delays, model uncertainties



# Rough terrain locomotion with heuristic strategies

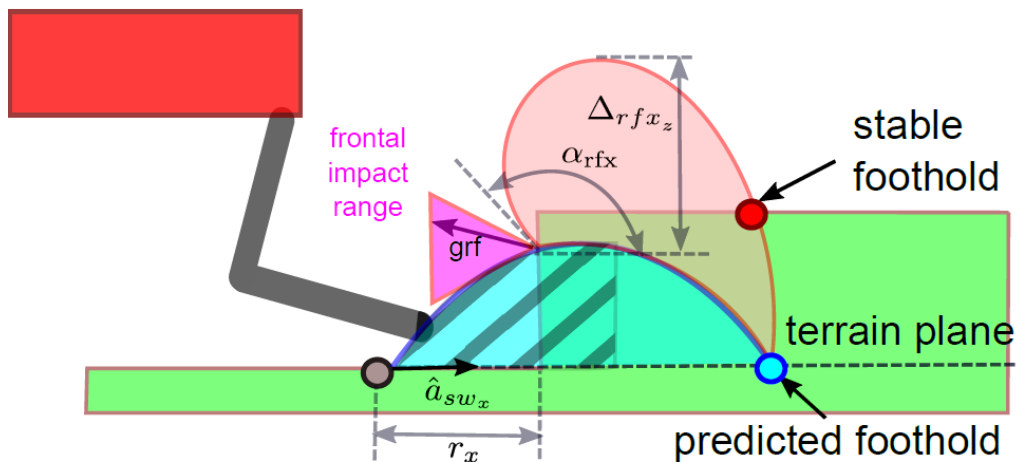
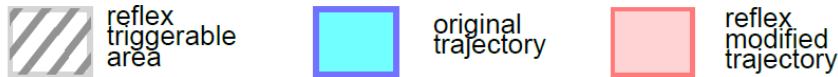
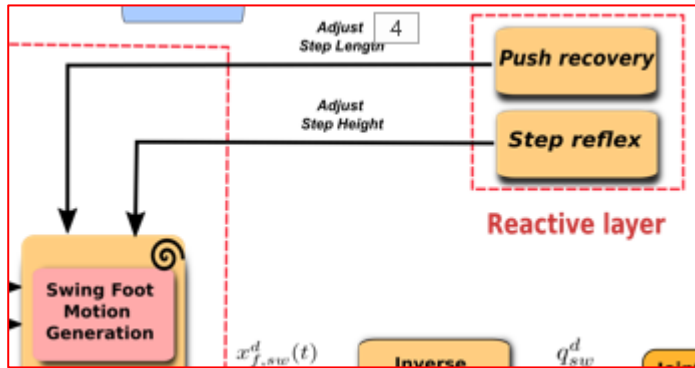
## IIT legged locomotion arena



HyQ (blindly) climbs up and down a ramp and navigates across an area with big-stones (diameter  $>10$  cm)

# Reactive Behaviours: Step Reflex

- Modify the swing motion to cope with unpredicted events



**STEP REFLEX OFF**



**STEP REFLEX ON**

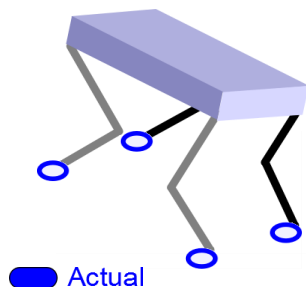


# State estimation in legged robots

- To close the loop the robot needs to know where he is in an inertial frame → **state estimation problem**
- By fusing different sources an EKF reduces the drift problem obtaining a more **accurate** estimation



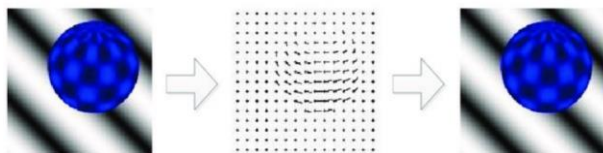
IMU



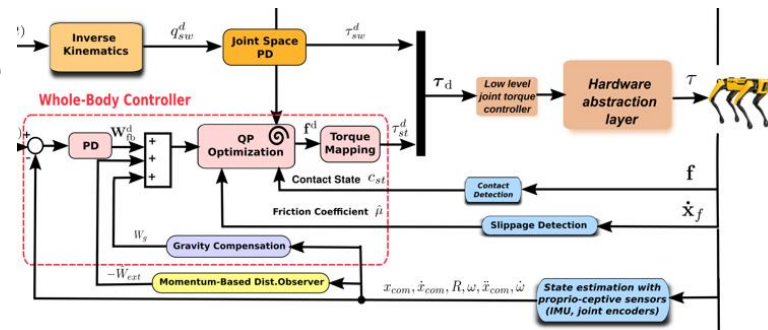
Leg Odometry  
(contact legs)



LiDar: scan matcher  
to correct drift in position



Visual Odometry



Extended  
Kalman  
Filter (EKF)  
Sensor Fusion

Estimates these  
robot states:  
linear position,  
linear velocity

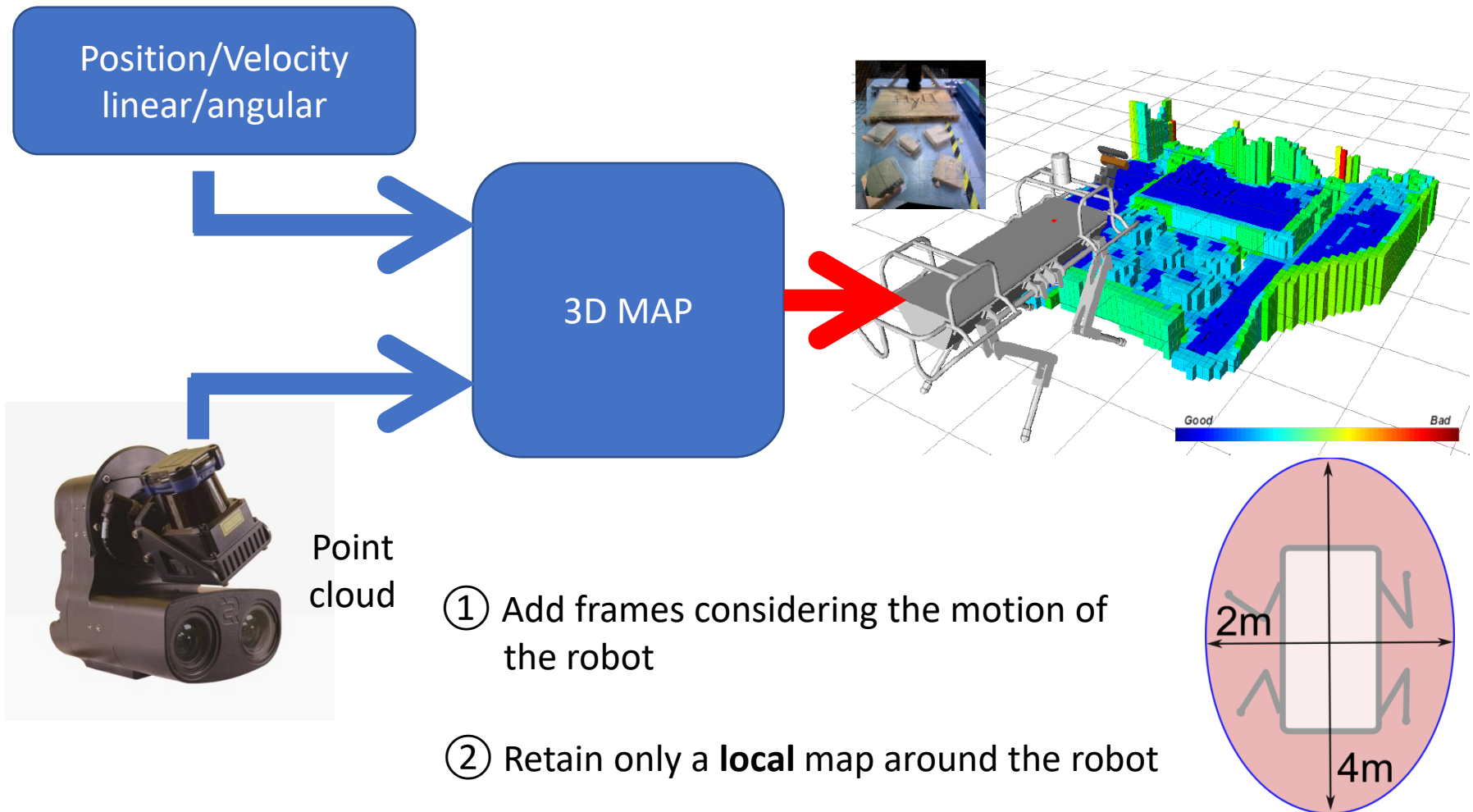
Intel T265



See the lecture of Marco Camurri!

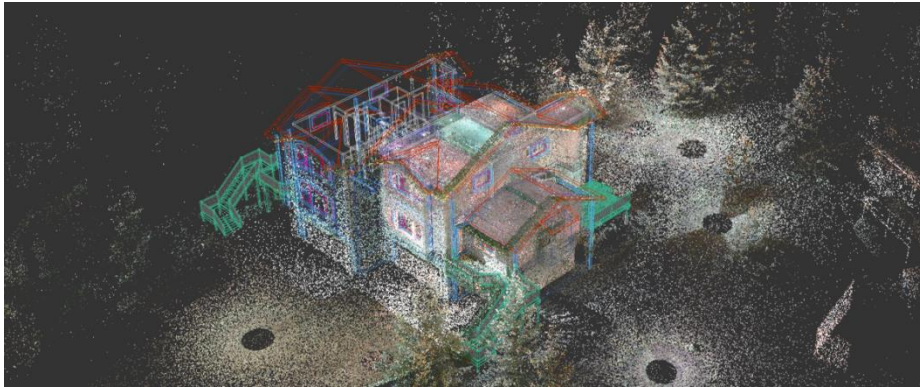
# Terrain Perception

- State estimation can be essential to create a global map of the environment



# Point cloud can be used for online mapping

- We have different representations that can be reconstructed starting from the acquired point cloud



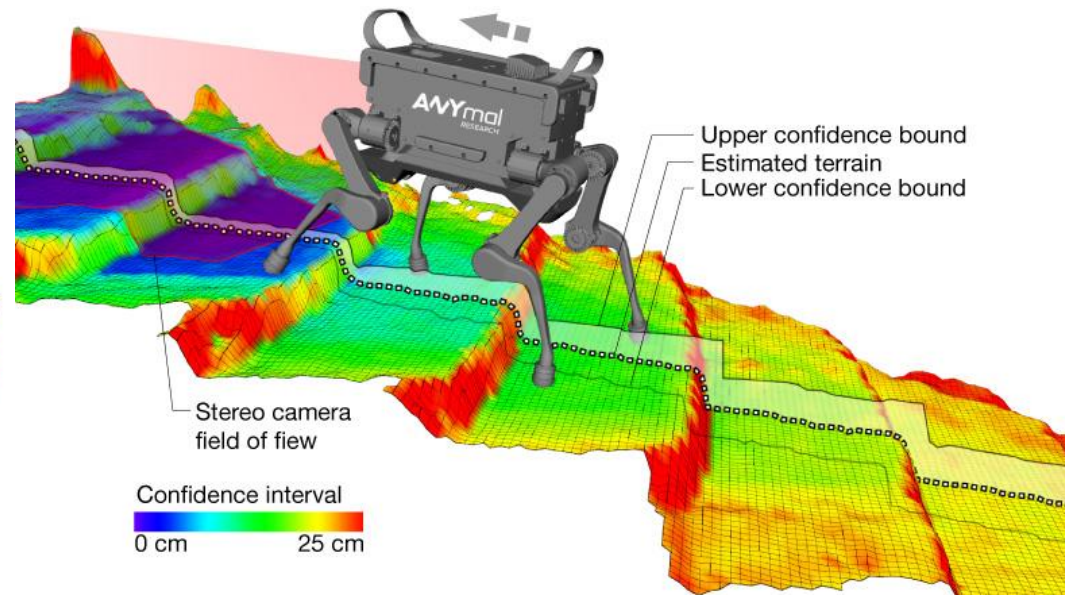
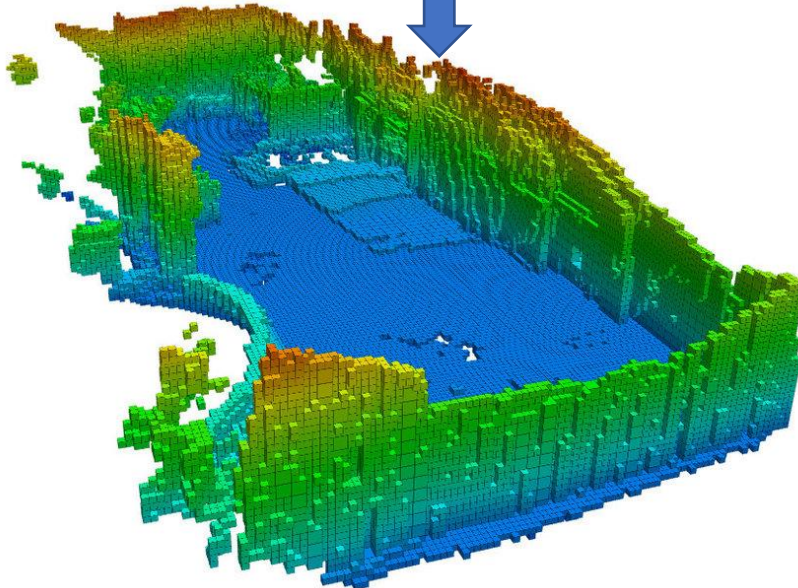
- **Height map** or elevation map (2.5D)



[https://github.com/ANYbotics/elevation\\_mapping](https://github.com/ANYbotics/elevation_mapping)

[https://github.com/leggedrobotics/elevation\\_mapping\\_cupy](https://github.com/leggedrobotics/elevation_mapping_cupy)

- **Octomap (3D), occupancy map**



# Local VS Global Map

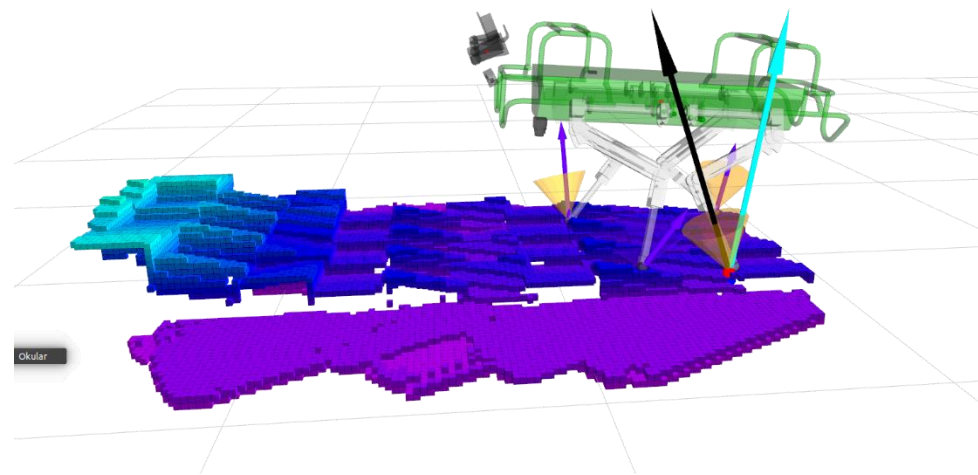


## Global Map:

- Usually acquired with LIDAR
- Used for navigation
- Possible to do loop closure (SLAM)

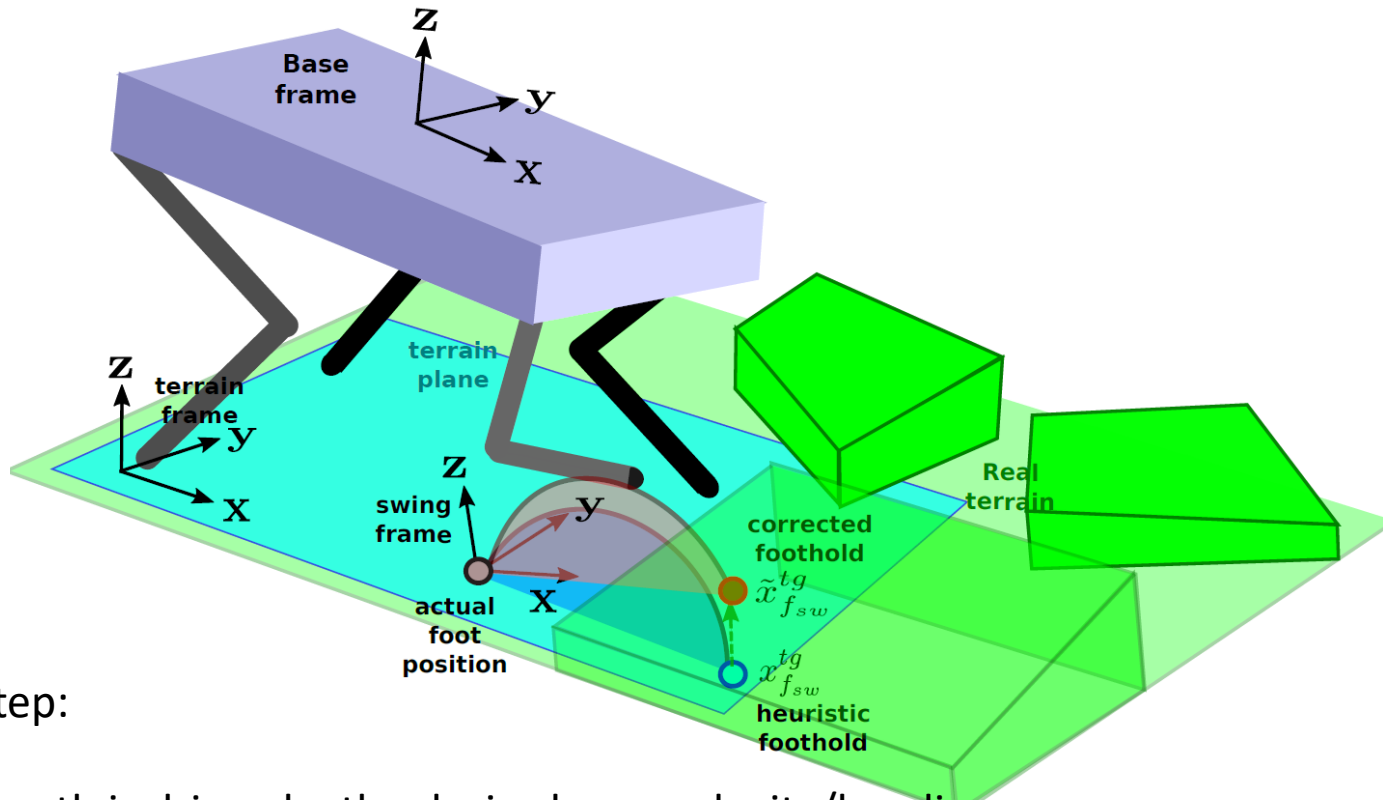
## Local Map:

- Foothold selection
- Dynamic obstacle avoidance



# Vision based stepping

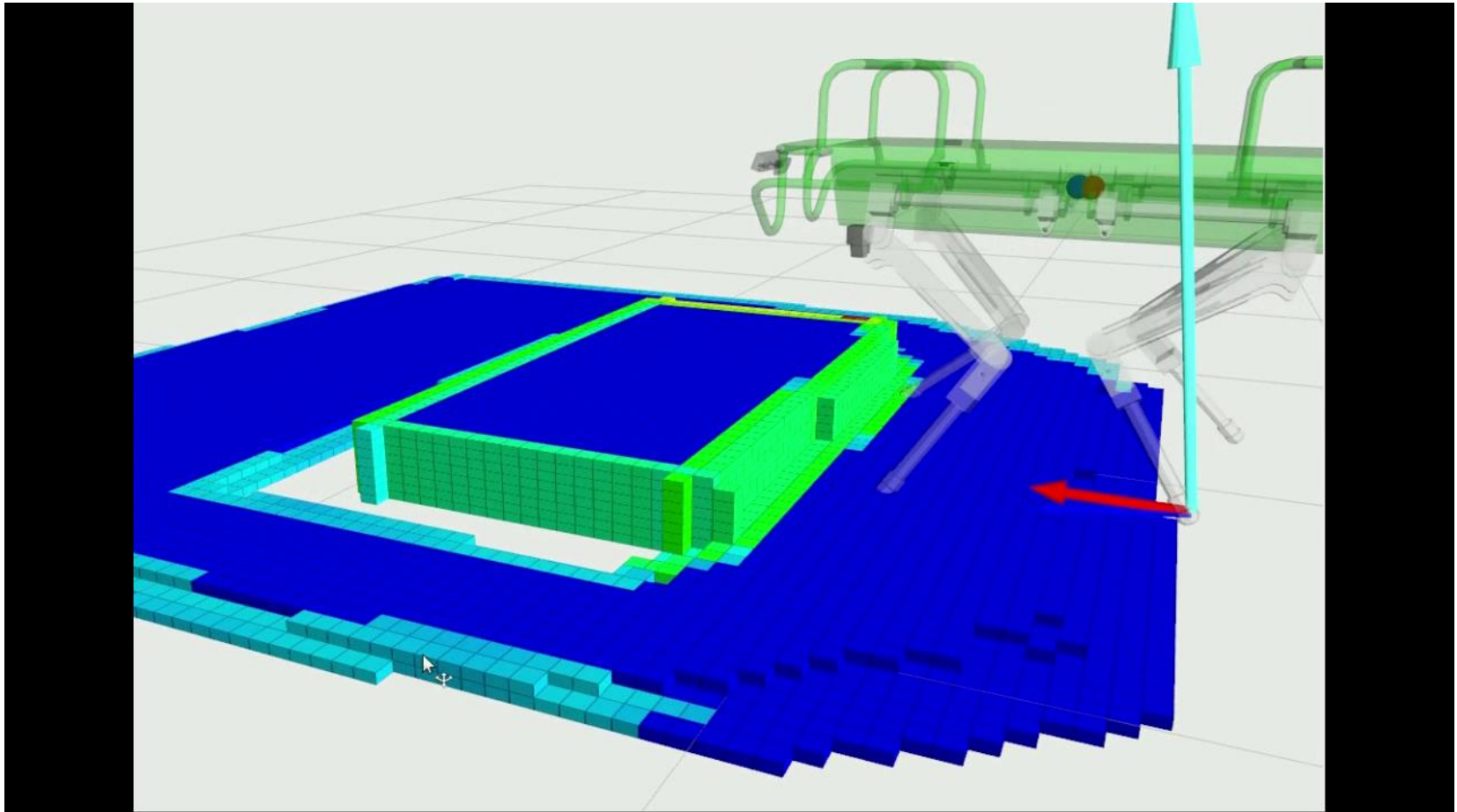
**IDEA:** Incorporate vision input to correct step height (vision enhanced locomotion)



for each step:

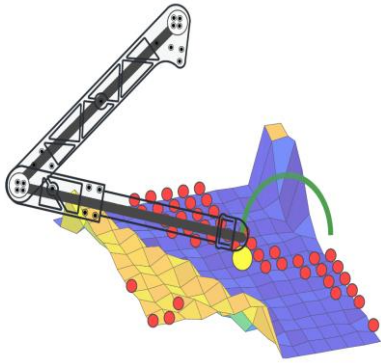
- Step length is driven by the desired user velocity/heading
- evaluate the height map at the heuristic location
- correct the foothold target to the real value of the terrain

## Vision based stepping experiment

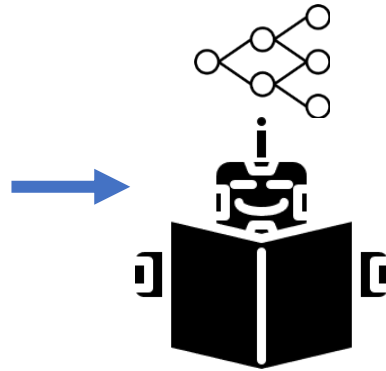


# Vision-based foothold adaptation

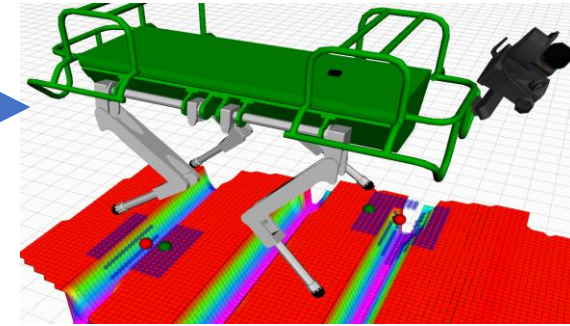
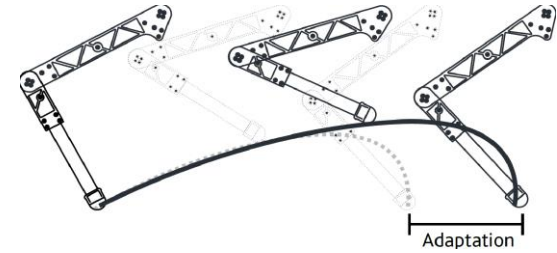
- Use map to Choose where to place your feet fast and safely



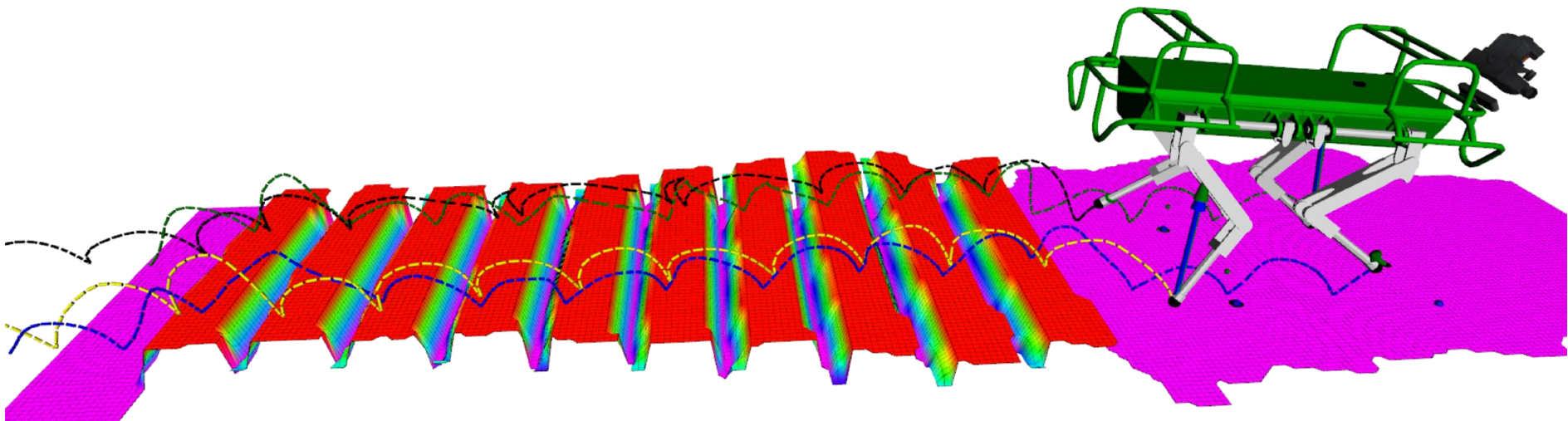
Heuristics



Neural network

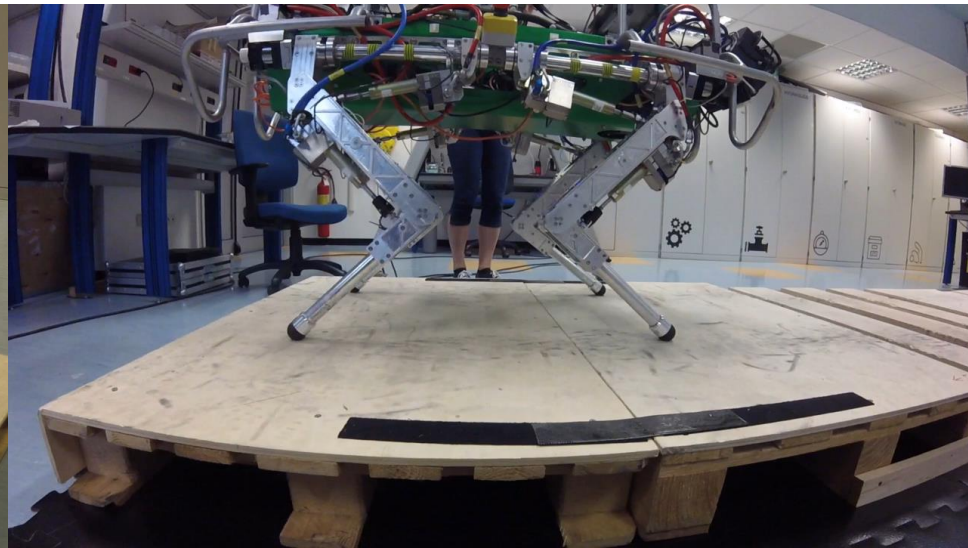
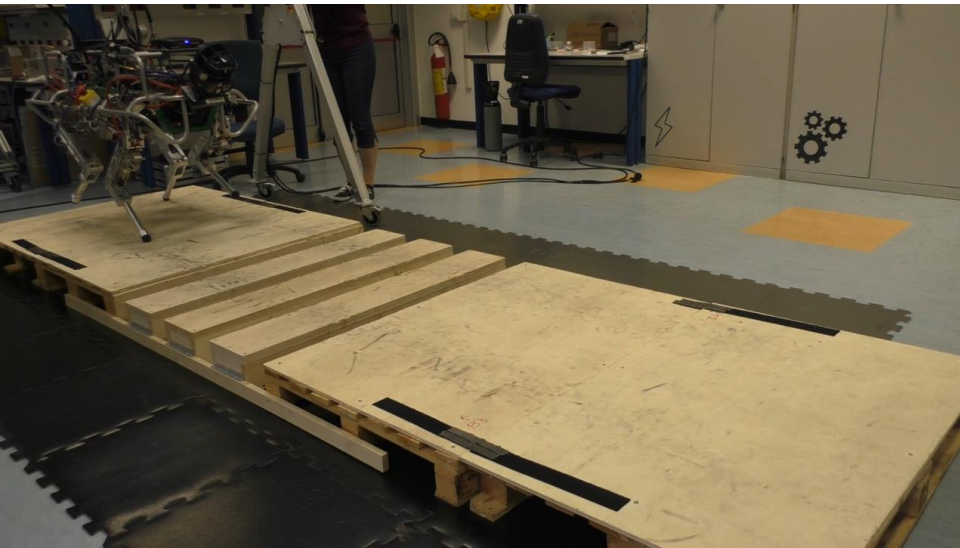


Computational gain: **15x to 200x**



# Vision-based foothold adaptations Experiments [O. Villarreal, IEEE RA-L, 2019]

Trotting at 0.5 m/s  
with dynamic visual reactions



# Instantaneous control VS optimal control

## Instantaneous control

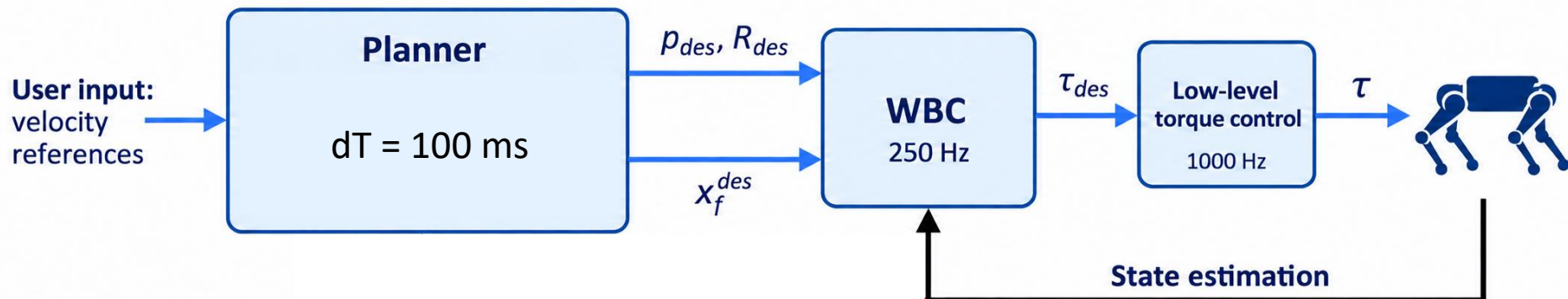
-  needs reference trajectory to track
-  Can handle constraints in the actual instant (friction/torque/velocity limits)
-  No knowledge of future states, difficulty to anticipate possible undesired events in the future

## Trajectory Optimization/Optimal control

-  No need of a reference trajectory (optional )
-  Objective clearly specified through cost function
-  Using models we can predict the future in order to fulfill constraints
-  can generate dynamically feasible trajectories.
-  computationally expensive:
  - e.g. 2s horizon at 1 ms  $\rightarrow$  2000 points;
  - 30 variables (velocity/torques)  $\times$  2000.

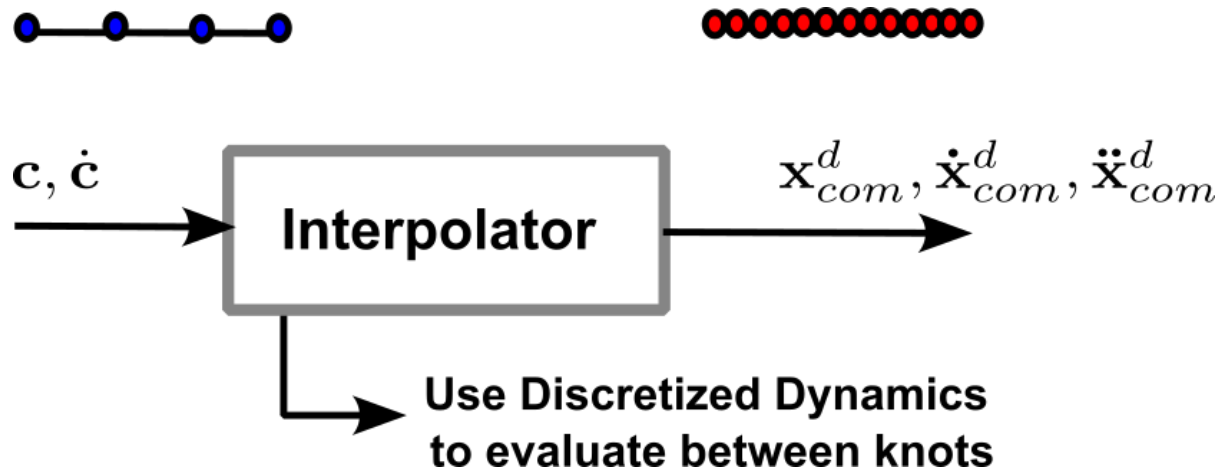
# Trajectory optimization

- Let's do trajectory optimization **offline** to obtain a **reference** trajectory. We can run it together with reactive controllers. The reference will be feasible and will satisfy constraints.
- Throughout the horizon the trunk controller/whole-body controller has to stabilize only around that trajectory.
- If the reference trajectory is sufficiently far from feasibility boundaries, the controller will be able to track even in face of disturbances.
- The controller runs at much higher frequency than the one associated to the plan discretization



# Closing the loop with the whole-body controller: interpolator

- Optimized trajectories have difference discretization (e.g. 100 ms VS 1 ms)
- To use a CoM reference as an input for the Whole-body Controller we need an **interpolator**



For LIP we have  
closed loop  
form!

$$\begin{bmatrix} c_x(t) \\ \dot{c}_x(t) \end{bmatrix} = \begin{bmatrix} \cosh(\omega t) & \frac{1}{\omega} \sinh(\omega t) \\ \omega \sinh(\omega t) & \cosh(\omega t) \end{bmatrix} \begin{bmatrix} c_{x_0} \\ \dot{c}_{x_0} \end{bmatrix} + \begin{bmatrix} 1 - \cosh(\omega t) \\ -\omega \sinh(\omega t) \end{bmatrix} z_x$$

$$\oplus$$

$$\ddot{c}_x = \omega^2 (cx - p_x)$$

# Computation Time in Trajectory Optimization

- Computation time is high because:
  - High #DoFs (30 for humanoids/12 for quadrupeds).
  - Non-convex (unless you linear models).
  - Contact optimization
    - Contacts sufficiently stiff to be modeled as rigid → hybrid system → non-smooth optimization sequence: jumps when you make/break contact → mixed integer problems
    - Soft contacts (but stiff): stiff differential equations → numerically ill-conditioned → integration slow.

**Solution:** use rigid contacts but **predefine contact sequence** → dynamics becomes **time-varying** (no longer hybrid)



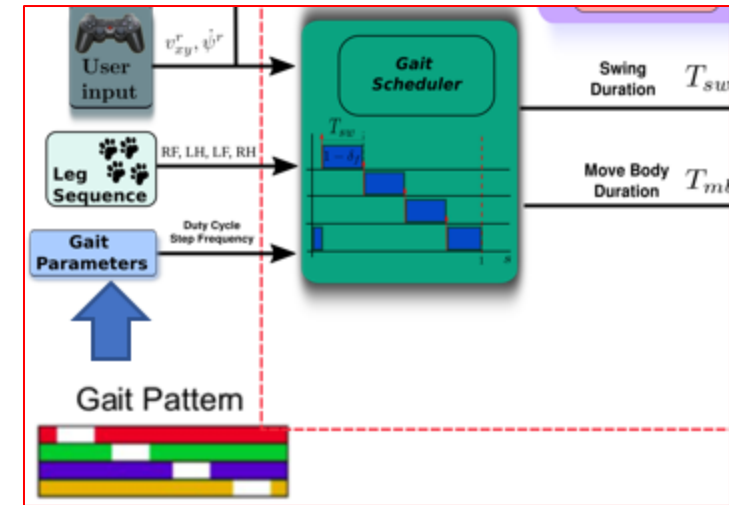
Use a **GAIT SCHEDULER**

Bipeds: easy, SS / DS / SS.

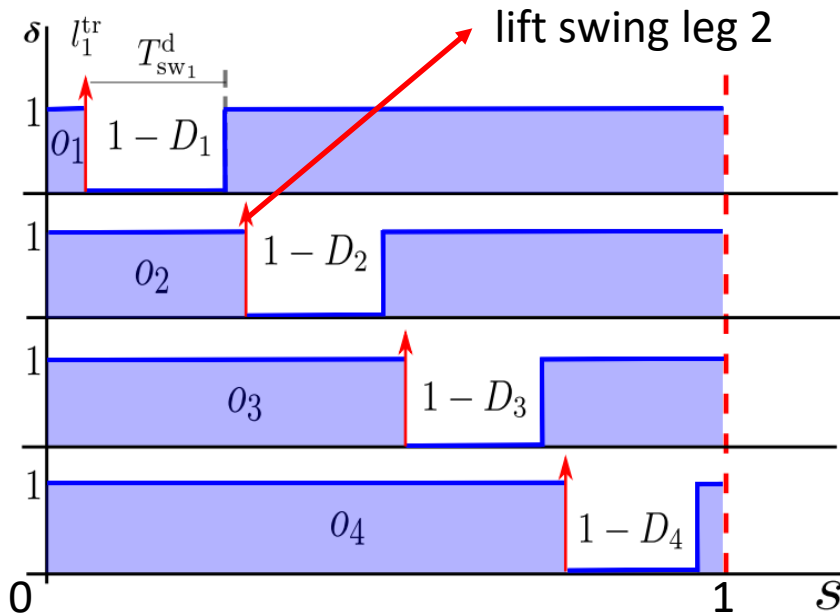
Quadrupeds: more complex.

# Gait scheduler

- A compact parametrization specifies which feet should support the robot throughout a gait cycle
- implements a stride parametrization  $s$  in  $[0, 1]$  normalized about the cycle time duration  $T_c$
- Each leg has a contact state:  
 $\delta_i(s)$ : 1 in stance       $\delta_i(s)$ : 0 in swing



## CRAWL EXAMPLE



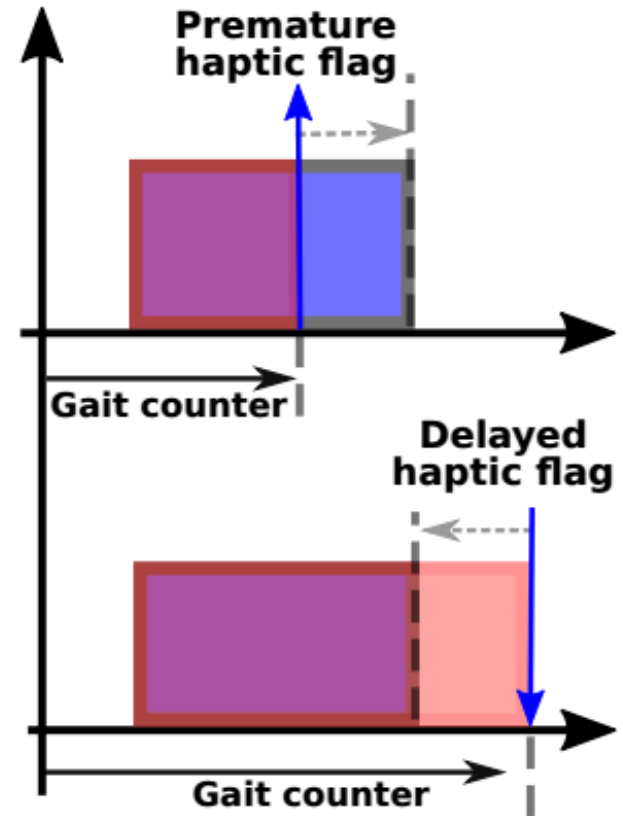
$$\delta_i = \begin{cases} 1, & s < o_i \vee s > ((o_i + (1 - D_i)) \bmod 1) \\ 0, & \text{otherwise} \end{cases}$$

- The offset  $o_i$  represents the moment in which the leg  $i$  begins a swing motion
- duty factor  $D_i$  indicates the time in which the leg  $i$  is in stance.  $D_i = \frac{T_{sw,i}}{T_c}$
- leg duty factor  $D_i$  and offsets  $o_i$  can be used to encode different gaits (e.g. crawl, trot)

Offsets  $o = [0.05, 0.3, 0.55, 0.8]$ , duty-factors  $D = [0.85, 0.85, 0.85, 0.85]$ .

# Adaptive Gait scheduler

- At each call, the reference generator extracts N contact states from the schedule, starting at the gait counter.
- In case of **premature** or **delayed** touch-down the counter of the gait scheduler must be fast-forwarded (top) or re-winded (bottom) to re-conciliate the planned touchdown with the actual touchdown
- Why synchronization matters? avoids commanding zero contact force while a foot has already on ground, or commanding a contact force for a foot that is still in the air.
- This is a crucial feature when dealing with rough terrains.



# Trajectory optimization and lower dimensional models

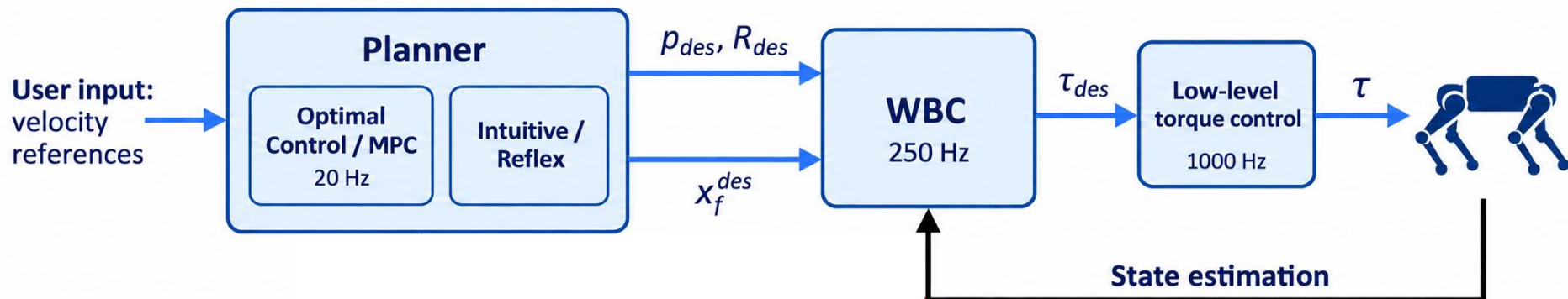
- Using a lower dimensional model for locomotion planning has several consequences
- The LIP reduced model does not capture joint related constraints. The only constraint (implicitly) considered is  $f_{z,i} > 0, \quad \forall i,$
- then other constraints are ignored.
  - There are no velocity bounds.
  - There are no self-collision constraints.
  - There are no kinematic limits.
  - There are no acceleration limits.
  - There are no torque limits.
- We can cope with these limits using more sophisticated models enforcing additional constraints but feasibility is difficult to be guaranteed because:
  - all models are wrong
  - disturbances are present
  - the controller has tracking errors;
  - parameters are difficult to model accurately (e.g. example the real friction coefficient)
  - Horizon is not infinite

the robot has to «give in» to perturbation, if keeps tracking the original trajectory it will fall

real constraints will never be fulfilled perfectly.

# Online re-planning

- Online motion generation is needed for reactivity and for maintaining stability in the long term.
- This moves the problem from classical optimal-control theory toward solving an optimization problem **every loop**
- At each loop we read the actual states and update the feasible set (e.g. obstacles relative position, need visual input) to **initialize** a new optimization
- This enables also to promptly accommodate to velocity set-point set by the user

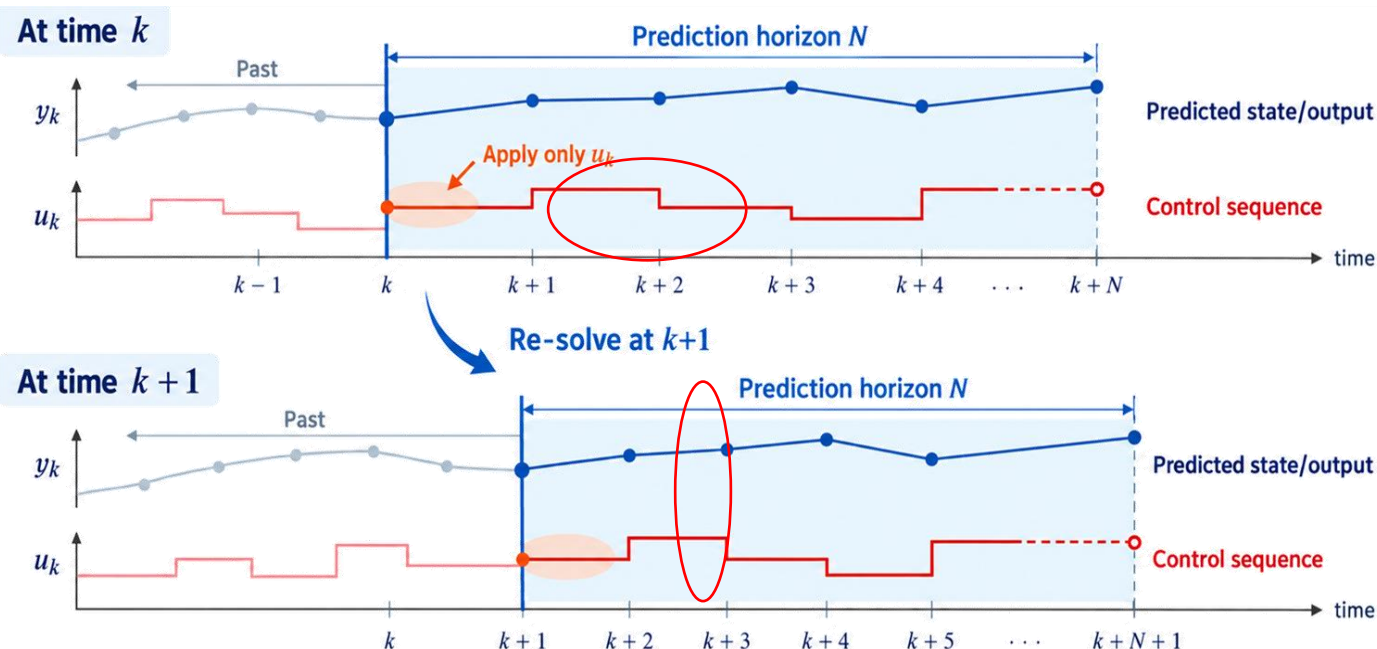


# Idea of Model Predictive control

- **Model Predictive Control (MPC)**: control scheme that repeatedly solve, online, a sequence of optimal control problems

$$\begin{aligned} \min_{\mathbf{u}(\cdot)} \quad & \int_t^{t+T} \ell(\mathbf{x}(\tau), \mathbf{u}(\tau)) d\tau + \ell_F(\mathbf{x}(t+T)) \\ \text{s.t.} \quad & \dot{\mathbf{x}}(\tau) = \mathbf{f}(\mathbf{x}(\tau), \mathbf{u}(\tau)), \quad \forall \tau \in [t, t+T], \\ & \mathbf{x}(t) = \mathbf{x}_{\text{init}}. \end{aligned}$$

- while accounting for the latest **available observation** of the system
- Only the first control action from the optimization  $\mathbf{u}(t)$  is applied to the system.
- At the next state observation, the horizon is **shifted** forward and the optimization is solved again (receding horizon)



- If the horizon ( $N$ ) is sufficiently large, can stabilize the system.
- larger horizon generally gives more opportunity to recover
- but increases the computational burden.

## MPC experimental results

[N. Rathod et al. 2021]



- MPC: quick adaptation to terrain changes (pallet relocation), can cope with state drifts and uncertainties

# Adaptive Footsteps

- Unless you specify with Raibert heuristic, foot placements can become a **decision variable**.
- This will increase the size of the optimization problem, hence the computation time
- adaptive foot placement is usually the more important additional action because it changes the support region improving balance performances against pushes
- The more time the robot has before the next touchdown, the more it can modify the foot placement.

See lecture of Enrico Mingos!